

Mini-projet 17 — Le scan asynchrone (asyncio)

Pourquoi ce mini-projet ?

Les threads (MP16) marchent, mais chaque thread coûte de la mémoire — au-delà de quelques centaines, ça sature. L'**asyncio** fait la même chose (plein d'attentes réseau en même temps) avec **un seul thread**, donc bien plus léger : tu peux lancer des **milliers** de connexions concurrentes. C'est le paradigme moderne du réseau en Python (utilisé par les gros scanners, serveurs web async...). C'est aussi le concept le plus déroutant du parcours — on y va tout doux.

1. L'idée : un seul worker qui ne reste jamais à attendre

L'image du maître d'échecs : un champion joue contre 20 personnes en simultané. Il joue un coup sur l'échiquier n°1, et au lieu d'**attendre** que l'adversaire réfléchisse, il passe au n°2, n°3... Quand il revient au n°1, l'adversaire a joué. **Un seul cerveau**, mais jamais inactif. L'asyncio, c'est ça : un seul thread qui, dès qu'une tâche **attend** (le réseau), passe à une autre.

2. Les 3 mots-clés

Mot-clé	Sens
<code>async def</code>	définit une coroutine : une fonction « pausable ». L'appeler ne l'exécute pas tout de suite.
<code>await</code>	« mets-moi en pause ici jusqu'à ce que cette opération async finisse — et pendant ce temps, laisse les autres tâches avancer ».
<code>asyncio.run(...)</code>	démarre la « boucle d'événements » (l'event loop) qui orchestre tout. C'est le point d'entrée.

```
import asyncio

async def bonjour():
    print("début")
    await asyncio.sleep(1)    # pause 1s, MAIS libère le thread pour d'autres tâches
    print("fin")

asyncio.run(bonjour())      # démarre la boucle et exécute la coroutine
```

Piège : appeler une coroutine sans `await` (`bonjour()`) ne l'exécute **pas** — ça crée juste un objet coroutine. Il faut toujours l' `await` , ou la passer à `asyncio.run` / `gather` .

3. Tester un port en async

asyncio fournit `open_connection(ip, port)` qui tente une connexion TCP. Si elle réussit → port ouvert. On l'entoure de `wait_for` pour le timeout.

```
async def port_ouvert_async(ip, port, timeout=1):
    try:
        reader, writer = await asyncio.wait_for(
            asyncio.open_connection(ip, port), timeout=timeout)
        writer.close()          # on referme la connexion
        await writer.wait_closed()
        return True             # connexion réussie = ouvert
    except (asyncio.TimeoutError, ConnectionRefusedError, OSError):
        return False            # timeout / refus / autre = fermé ou filtré
```

- `asyncio.wait_for(coro, timeout=...)` = « exécute cette coroutine, mais abandonne après `timeout` secondes » (lève `asyncio.TimeoutError`).
- `open_connection` renvoie **2 objets** (un lecteur, un écrivain) ; on n'a pas besoin de lire, on ferme juste `writer` .

4. Lancer 1000 tests EN MÊME TEMPS : asyncio.gather

`gather` prend plusieurs coroutines et les fait tourner **concurrentement**, puis renvoie la liste de leurs résultats **dans l'ordre**.

```

async def scan_async(ip, ports, timeout=1):
    taches = [port_ouvert_async(ip, p, timeout) for p in ports] # une coroutine par port
    resultats = await asyncio.gather(*taches) # TOUTES en même temps, attend tout
    return [p for p, ouvert in zip(ports, resultats) if ouvert]

ouverts = asyncio.run(scan_async("127.0.0.1", list(range(8000, 8101))))
print(ouverts) # [8080]

```

Les nouveautés à noter :

- `[port_ouvert_async(ip, p, timeout) for p in ports]` : une compréhension qui crée une **liste de coroutines** (elles ne tournent pas encore — pas d' `await`).
- `*taches` : l'**étoile** déballe la liste en arguments séparés pour `gather` (c'est le `*args` du MP02, à l'appel cette fois).
- `zip(ports, resultats)` : `gather` renvoie les résultats **dans l'ordre des ports** → `zip` réassocie chaque port à son booléen. (Pas besoin de `sorted` ici, l'ordre est préservé.)

5. Threads vs asyncio — lequel quand ?

	Threads (MP16)	asyncio (MP17)
Modèle	plusieurs threads système	un seul thread, une event loop
Coût mémoire	lourd (limite ~centaines)	léger (des milliers OK)
Écriture	plus simple (fonctions normales)	faut penser <code>async</code> / <code>await</code> partout
Idéal pour	quelques centaines de tâches I/O	très grand nombre de connexions

Les deux sont de bons choix. Pour un scanner de milliers de ports, asyncio est plus élégant et scalable. Savoir faire les deux, c'est ce qu'un recruteur veut voir.

🎯 Ta mission

Objectif : refaire ton scanner, version asyncio.

1. Écris `async def port_ouvert_async(ip, port, timeout=1)` qui renvoie `True` / `False` (via `open_connection` + `wait_for` , avec le `try/except`).
2. Écris `async def scan_async(ip, ports, timeout=1)` qui crée une coroutine par port, les lance avec `gather` , et renvoie la liste des ports ouverts.
3. Lance le tout avec `asyncio.run(...)` .

Tests (serveur `http.server 8080` lancé) :

```

ports = list(range(8000, 8101))
ouverts = asyncio.run(scan_async("127.0.0.1", ports))
print(ouverts) # -> [8080]

import time
debut = time.time()
tout = asyncio.run(scan_async("127.0.0.1", list(range(1, 1001))))
print(f"{tout} en {time.time()-debut:.2f}s")

```

Bonus : lance un scan sur une **grosse** plage (ex. 1 à 5000) et constate que ça reste rapide et léger, là où 5000 threads feraient souffrir ta machine.

✓ Critères de réussite

- `asyncio.run(scan_async("127.0.0.1", list(range(8000, 8101))))` renvoie `[8080]` .
- `port_ouvert_async` est bien une `async def` et utilise `await` .
- Tu utilises `asyncio.gather` pour lancer les tests concurremment.
- Aucune coroutine « oubliée » (chacune est `await` -ée, directement ou via `gather`).

Si tu bloques — indices progressifs

Étape	Indice
-------	--------

Import	<code>import asyncio</code> en haut.
Tester un port	<code>await asyncio.wait_for(asyncio.open_connection(ip, port), timeout=timeout)</code> dans un <code>try</code> . Si ça passe → <code>return True</code> . <code>except (asyncio.TimeoutError, OSError): return False</code> .
Fermer proprement	Après connexion : <code>writer.close()</code> puis <code>await writer.wait_closed()</code> .
Lancer en masse	<code>taches = [port_ouvert_async(ip, p, timeout) for p in ports]</code> puis <code>resultats = await asyncio.gather(*taches)</code> (n'oublie pas l'étoile <code>*</code>).
Filtrer les ouverts	<code>[p for p, ok in zip(ports, resultats) if ok]</code> .
Démarrer	Tout en bas, hors des <code>async def</code> : <code>asyncio.run(scan_async(...))</code> .

Où ranger : `mini-projets/17-asyncio/scan_async.py`.

Note : `await` ne s'utilise QUE dans une `async def`. Si tu écris `await` dans une fonction normale → `SyntaxError`. C'est le point qui piège au début.

Quand c'est prêt : `git add` . → `commit` → `push` , puis « c'est poussé ». Ce sera la **fin du Module D** — après, on assemble le grand projet !